So what do you do when a user wants to edit a message in a conversation? You can send an `EditMessage` message to the conversation. The `EditMessage` contains a modified copy of the message instead of updating the message in place in a shared messages list. The `Conversation` actor receives the `EditMessage` and replaces the existing message with the new copy.

Immutability is an absolute necessity when it comes to concurrency. It is another restriction that simplifies life because there are fewer moving parts to manage.

The order of sent messages is kept between a sending actor and a receiving actor. An actor receives messages one at a time. Imagine that a user edits a message many times; it would make sense for the user to eventually see the result of the final edit of the message. The order of messages is only guaranteed per sending actor, so if many users edit the same message in a conversation, the final result can vary depending on how the messages are interleaved over time.

Create

An actor can create other actors. Figure 1.17 shows how the `Supervisor` actor creates a `Conversations` actor. As you can see, this action automatically creates a hierarchy of actors. The chat application first creates the `Supervisor` actor, which in turn creates all other actors in the application. The `Conversations` actor recovers all `Conversation`s from the journal. It then creates a `Conversation` actor for every `Conversation` that recovers itself from the journal.